MODULE *Exokernel*

EXTENDS *Naturals*, *Sequences*

Parameters and constants

CONSTANT *TaskCount*, *ResourceCount*

VARIABLE *exokernelState*

TaskId are modeled as a sequence of task *IDs*, where each task has an index.

$TaskId \triangleq 1 \,..\, TaskCount$

ResourceId are modeled as a sequence of resource *IDs*, where each resource has an index.

$ResourceId \triangleq 1 \,..\, ResourceCount$

$TaskState \triangleq \{\text{“waiting”}, \text{“running”}\}$

$Availability \triangleq \{\text{TRUE}, \text{FALSE}\}$

Type invariant of the specification

exokernelState: record with fields tasks, resources, and allocations

$$
\begin{aligned}
&ExokernelTypeInvariant \triangleq \\
&\quad \land\ exokernelState.tasks \in [TaskId \to TaskState] \\
&\quad \land\ exokernelState.resources \in [ResourceId \to Availability] \\
&\quad \land\ exokernelState.allocations \in [TaskId \to [ResourceId \to \{\text{TRUE}, \text{FALSE}\}]]
\end{aligned}
$$

Define *Exokernel* state as a record containing tasks, resources, and allocations

$ExokernelInit \triangleq exokernelState = [tasks \mapsto [t \in TaskId \mapsto \text{“waiting”}], resources \mapsto [r \in Resource$

A task can request access to a resource if it's not already in use.

$$
\begin{aligned}
&RequestResource(t, r) \triangleq \\
&\quad \land\ exokernelState.resources[r] = \text{TRUE} \\
&\quad \land\ exokernelState.allocations[t][r] = \text{FALSE} \\
&\quad \land\ exokernelState.tasks[t] = \text{“waiting”} \\
&\quad \land\ exokernelState' = \\
&\qquad [tasks \mapsto [exokernelState.tasks \text{ EXCEPT } ![t] = \text{“running”}], \\
&\qquad\ resources \mapsto [exokernelState.resources \text{ EXCEPT } ![r] = \text{FALSE}], \\
&\qquad\ allocations \mapsto [exokernelState.allocations \text{ EXCEPT } ![t][r] = \text{TRUE}] \\
&\qquad]
\end{aligned}
$$

A task can release a resource when it has finished using it.

$ReleaseResource(t,\ r) \triangleq$
$\land\ exokernelState.allocations[t][r] = \text{TRUE}$
$\land\ exokernelState.tasks[t] = \text{“running”}$
$\land\ exokernelState' =$
$[tasks \mapsto [exokernelState.tasks \text{ EXCEPT } ![t] = \text{“waiting”}],$
$resources \mapsto [exokernelState.resources \text{ EXCEPT } ![r] = \text{TRUE}],$
$allocations \mapsto [exokernelState.allocations \text{ EXCEPT } ![t][r] = \text{FALSE}]$
$]$

Task behavior: tasks can either request or release resources
$TaskAction \triangleq$
$\exists\, t \in TaskId,\ r \in ResourceId :$
$(RequestResource(t,\ r) \lor ReleaseResource(t,\ r))$

The next-state relation defines valid state transitions
$ExokernelNext \triangleq TaskAction$

Specification of the overall system behavior
$ExokernelSpec \triangleq ExokernelInit \land \Box[ExokernelNext]_{exokernelState}$

$vars \triangleq \langle exokernelState \rangle$

Fairness $\triangleq$ WF_$vars(TaskAction)$

Invariant: No two tasks can hold the same resource simultaneously
$ResourceExclusivity \triangleq$
$\forall\, r \in ResourceId :$
$\forall\, t1,\ t2 \in TaskId : (t1 \# t2) \Rightarrow \neg(exokernelState.allocations[t1][r] \land exokernelState.allocations[t2][r])$

ASSUME Assumption $\triangleq exokernelState.tasks \in [TaskId \to TaskState]$

THEOREM $Init \Rightarrow$ Fairness
BY DEF $Init$, Fairness
